

LOOP (PERULANGAN) DALAM R

Pertemuan 5

Weksi Budiaji & Ferdian Bangkit Wijaya

Arsitektur Dasar Pemrograman



Sequence (Berurutan)

Perintah dijalankan satu per satu dari atas ke bawah.

```
x <- 5  
y <- x + 1
```



Selection (Pemilihan)

Memilih perintah yang dijalankan berdasarkan suatu nilai.

```
if, else, switch, ifelse
```



Repetition (Perulangan)

Menjalankan blok perintah berulang kali.
Menggabungkan pengujian nilai dengan eksekusi.

```
for, repeat, while
```


Anatomi Sebuah Loop



Infinite Loop: Terjadi bila kondisi berhenti tidak pernah tercapai (misal: lupa menyertakan Update).

Inisialisasi: Pemberian nilai awal variabel sebelum loop dimulai.

```
iterate <- 1
```

Iterasi: Satu kali putaran eksekusi eksekusi badan loop.

```
while (iterate <= 5) {  
  print('Proses data')  
  iterate <- iterate + 1  
}
```

Kondisi: Ekspresi TRUE/FALSE yang menentukan loop lanjut atau berhenti.

Badan Loop (Body): Perintah di dalam tanda kurung kurawal yang dieksekusi berulang kali.

Update: Perubahan nilai variabel agar kondisi akhirnya menjadi FALSE dan loop berhenti.

Persiapan Toolset: Modulo & Fungsi

Operator Modulo (%)

Menghasilkan sisa bagi pembagian.

8 %% 2 =	0 (Genap)
5 %% 2 =	1 (Ganjil)
10 %% 4 =	2 (dibagi 4 sisa 2)

Fungsi angka2()

```
angka2 <- function(nilai) {  
  sisa <- nilai %% 2  
  hasil <- switch(sisa + 1, 'Genap', 'Ganjil')  
  return(hasil)  
}
```

⚠ Catatan Kritis: angka2() HANYA menerima **SATU** nilai per pemanggilan. Untuk memproses banyak angka, kita **WAJIB** menggunakan **loop**!

Target Vektor 'a'

```
set.seed(2023)  
a <- sample(1:10, 5, replace = TRUE)
```

Output a: [1] 5, 9, 8, 3, 10

Ini adalah vektor 'a' yang akan kita telusuri (trace) pada materi selanjutnya.

FOR Loop: Iterasi Terukur



```
for (variabel in urutan) {  
    pernyataan  
}
```

Gunakan saat jumlah putaran sudah diketahui pasti (iterasi terukur).

Kuadrat:

```
Grid  
for (i in 1:5) {  
    print(i^2)  
}
```

Akses Langsung:

```
Grid  
for (b in buah) {  
    print(b)  
}
```

Akumulator:

```
Grid  
total <- 0;  
for(i in 1:length(v)) {  
    total <- total + v[i]  
}
```

Kondisi:

```
Grid  
if (angka2(a[i]) == 'Genap') {  
    banyak <- banyak + 1  
}
```


Eksekusi FOR Loop (Jejak Variabel)

Kode

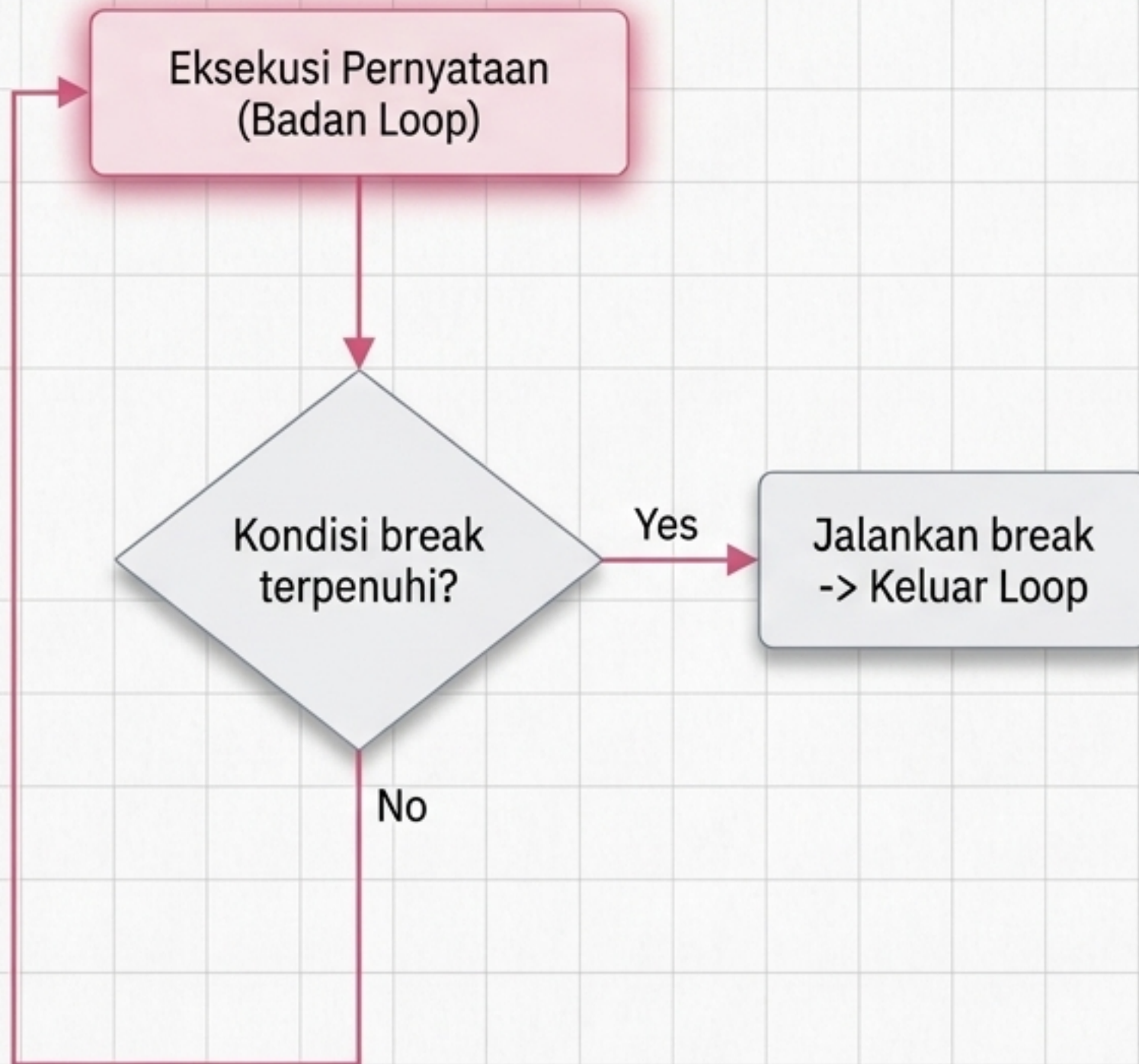
```
a <- c(5, 9, 8, 3, 10)
for (i in 1:length(a)) {
  print(angka2(a[i]))
}
```

Jejak Variabel (Trace Table)

Iterasi	i	a[i]	a[i] %% 2	sisas+1	Output print()
1	1	5	1	2	Ganjil
2	2	9	1	2	Ganjil
3	3	8	0	1	Genap
4	4	3	1	2	Ganjil
5	5	10	0	1	Genap

Catatan: Setelah $i = 5$ selesai, urutan 1:5 habis, sehingga loop otomatis berhenti.

REPEAT Loop: Berulang Tanpa Batas



```
repeat {  
  pernyataan  
  if (kondisi) { break }  
}
```

Mengulang tanpa batas. Pasti berjalan minimal 1 kali. Wajib memiliki perintah 'break' untuk mencegah infinite loop.

Contoh: Akumulator dengan Limit

```
n <- 0; total <- 0  
repeat {  
  n <- n + 1; total <- total + n  
  if (total > 20) { break }  
}
```

Output: n = 6, total = 21

Eksekusi REPEAT Loop (Jejak Variabel)

Kode

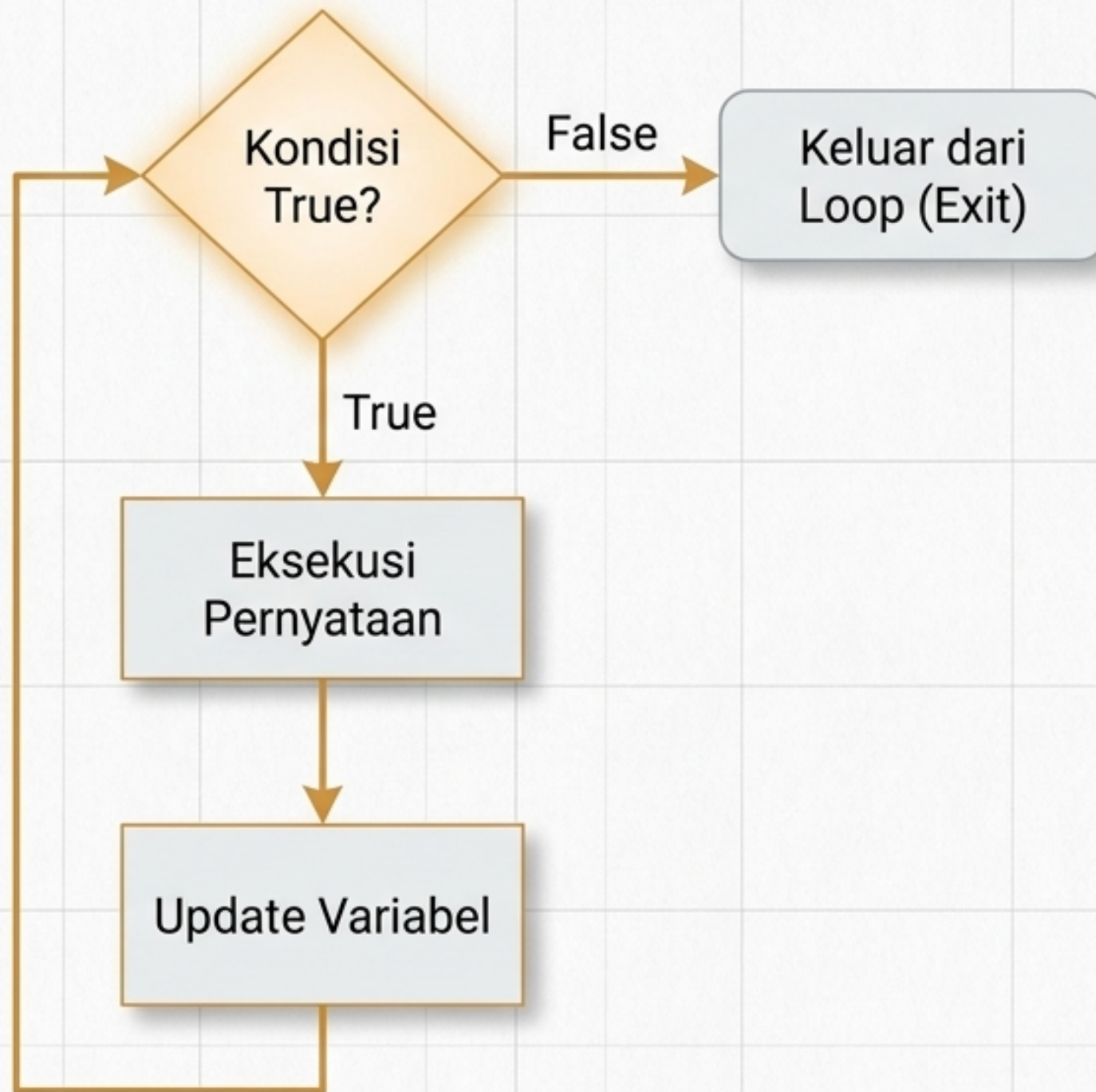
```
i <- 0
repeat {
  iterate <- i + 1
  print(angka2(a[iterate]))
  if (iterate == length(a)) {break}
  i <- iterate
}
```

Jejak Variabel (Trace Table)

Iterasi	i (awal)	iterate	a[iterate]	Output	iterate == 5 ?	i (akhir)
1	0	1	5	Ganjil	FALSE	1
2	1	2	9	Ganjil	FALSE	2
3	2	3	8	Genap	FALSE	3
4	3	4	3	Ganjil	FALSE	4
5	4	5	10	Genap	TRUE -> break	4 (tidak diupdate)

Catatan: Perhatikan baris terakhir. Karena perintah break dieksekusi, kode `i <- iterate` di bawahnya tidak pernah dijalankan.

WHILE Loop: Selama Kondisi Terpenuhi



```
while (kondisi) {  
    pernyataan  
    update_variabel  
}
```

Mengulang selama kondisi TRUE. Diperiksa di awal (gatekeeper). Wajib ada update variabel agar tidak menjadi infinite loop.

Hitung Mundur

```
n <- 3;  
while (n > 0) {  
    print(n);  
    n <- n - 1  
}
```

0 Iterasi (Kondisi False Sejak Awal)

```
n <- 0;  
while (n > 0) {  
    print('Tidak tampil')  
}
```


Eksekusi WHILE Loop (Jejak Variabel)

Kode

```
iterate <- 1
while (iterate < (length(a) + 1)) {
  print(angka2(a[iterate]))
  iterate <- iterate + 1
}
```

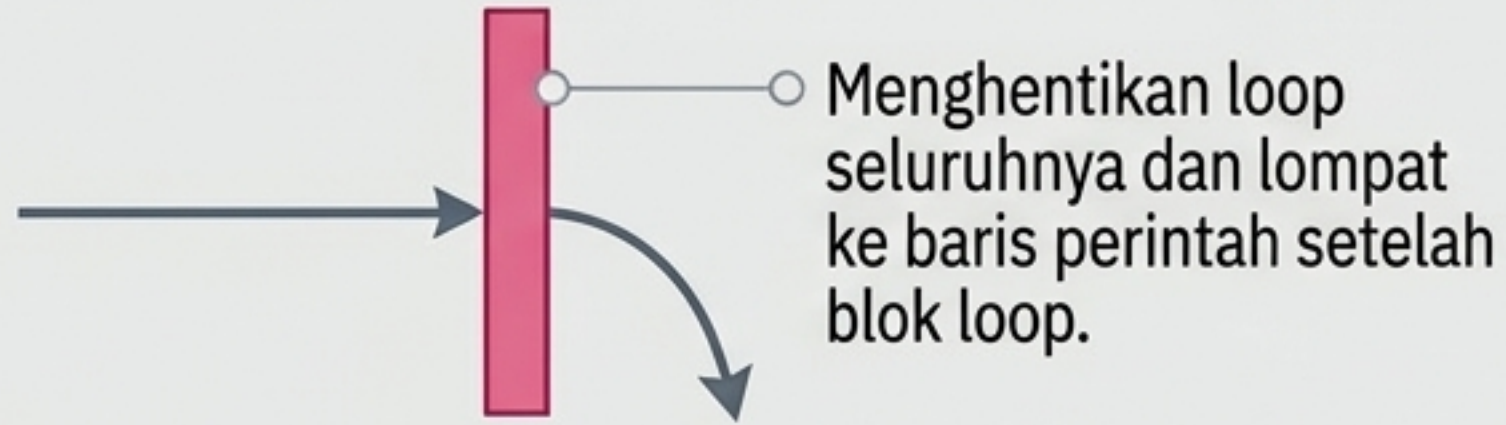
Jejak Variabel (Trace Table)

Iterasi	iterate (awal)	iterate < 6 ?	a[iterate]	Output	iterate (akhir)
1	1	TRUE	5	Ganjil	2
2	2	TRUE	9	Ganjil	3
3	3	TRUE	8	Genap	4
4	4	TRUE	3	Ganjil	5
5	5	TRUE	10	Genap	6
(cek)	6	FALSE	-	-	keluar loop

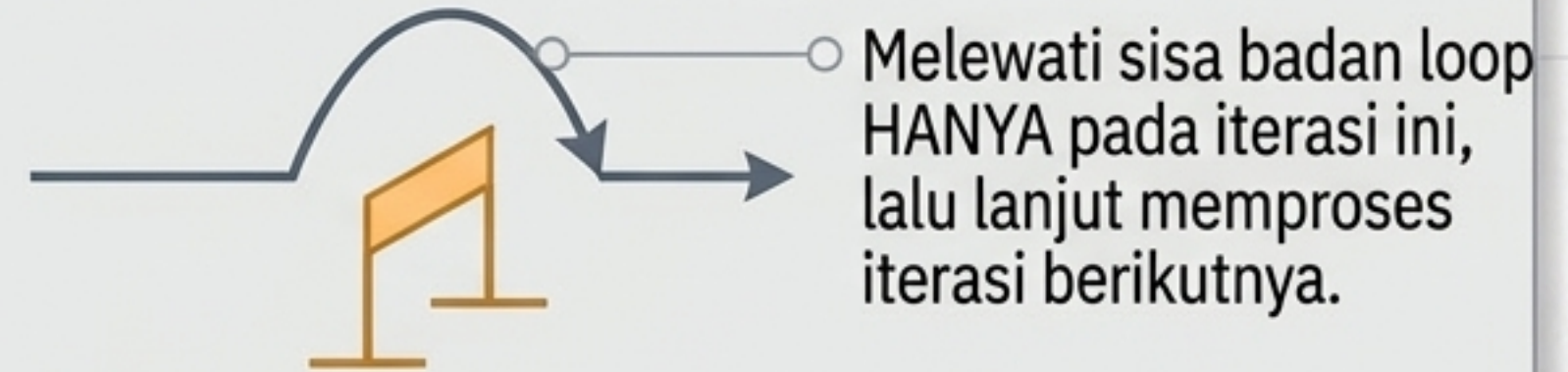
Catatan: Iterasi ke-5 menghasilkan $iterate = 6$. Pada pengecekan berikutnya, kondisi $6 < 6$ bernilai FALSE, sehingga loop dicegat di pintu masuk dan langsung berhenti.

Kendali Navigasi: BREAK vs. NEXT

BREAK (Menghentikan Total)



NEXT (Melompati Iterasi)



Aplikasi Jejak Kode (Code Trace Application)

```
for (i in 1:6) {  
  if (i == 3) next  
  if (i == 5) break  
  print(i)  
}
```



Final Output: 1, 2, 4

Matriks Perbandingan Tipe Loop (Sintesis)

	FOR	REPEAT	WHILE
Kapan Dipakai?	Iterasi terukur. Jumlah putaran sudah diketahui.	Berhenti ditentukan di tengah atau di akhir badan loop.	Jumlah tidak diketahui, berhenti berdasar kondisi di awal.
Pemeriksaan Kondisi	Otomatis (saat urutan habis).	Manual (menggunakan if + break).	Di awal (sebelum tiap iterasi dimulai).
Eksekusi Minimal	0 kali (jika urutan kosong).	Minimal 1 kali (pasti jalan dulu).	0 kali (jika kondisi FALSE sejak awal).
Pengaturan Update	Otomatis diatur oleh urutan.	Manual (harus ditulis eksplisit).	Manual (harus ditulis eksplisit).
Risiko Infinite Loop	Sangat kecil.	Tinggi (jika lupa menulis break).	Tinggi (jika lupa menulis kode update).

Ketiga tipe loop dapat menyelesaikan masalah logika yang sama. Perbedaannya hanya terletak pada mekanisme kontrol penghitung dan penghentian.

Jebakan Umum (Pitfalls) & Praktik Terbaik

Off-by-one Error

Salah memakai operator `<` vs `<=`. Menyebabkan elemen terakhir terlewat atau mengakses indeks kosong (menghasilkan NA).

↓

```
for(i in 1:length(x))  
vs  
for(i in 1:(length(x)-1))
```

Lupa Update Variabel

Variabel kondisi pada "while" tidak diubah di dalam blok. Kondisi selalu TRUE → Infinite Loop.

↓

```
while (i < 10) {  
  print(i)  
} # Missing i <- i + 1
```

Salah Inisialisasi

Variabel seperti "total" dimulai dari nilai yang salah, atau lupa dibuat sebelum masuk loop.

↓

```
total <- 1 # Wrong start  
for(x in vec) {  
  total <- total + x  
} # Error: total not found
```

Vektor Hasil Tak Disiapkan

Menambah elemen satu per satu ke dalam vektor memperlambat memori. (Praktik baik: Buat wadah kosong dulu).

↓

```
res <- c();  
for(i in 1:n)  
  res <- c(res, f(i)) # Slow  
vs  
res <- numeric(n);  
for(i in 1:n)  
  res[i] <- f(i) # Fast
```

Loop Vektor Kosong

Menggunakan `1:length(x)` saat x kosong akan memicu iterasi 1 lalu 0. (Gunakan `seq_along(x)` sebagai cara aman).

↓

```
x <- c();  
for(i in 1:length(x))  
  # Iterates 1, 0  
vs  
for(i in seq_along(x))  
  # Safe, no iteration
```

Pro-Tip: Loop vs Vektorisasi

R dirancang secara native untuk operasi vektor. Seringkali pekerjaan loop dapat ditulis lebih cepat secara vektorisasi. Contoh: `sum(v)` lebih baik dari loop akumulator. `ifelse(a %% 2 == 0, 'Genap', 'Ganjil')` lebih cepat dari loop klasifikasi. Namun, loop fundamental tetap vital untuk algoritma kompleks yang bergantung pada hasil iterasi sebelumnya.

Studi Kasus: Inversi Ganjil-Genap

Tugas: Mengubah kelas elemen vektor (Ganjil -> Genap, Genap -> Ganjil).

Ida Kunci: Menambah 1 pada bilangan bulat selalu membalik paritas kelasnya (misal 5 -> 6, 8 -> 9).

Kode inti: $y[i] \leftarrow x[i] + 1$

Solusi menggunakan FOR

```
for (i in 1:length(x)) {  
  y[i] <- x[i] + 1  
}
```

Solusi menggunakan REPEAT

```
i <- 0;  
repeat {  
  i <- i + 1;  
  if (i > length(x)) {  
    break  
  };  
  y[i] <- x[i] + 1  
}
```

Solusi menggunakan WHILE

```
i <- 1;  
while (i <= length(x)) {  
  y[i] <- x[i] + 1;  
  i <- i + 1  
}
```

Input: [5, 9, 8, 3, 10]` ----> Output: [6, 10, 9, 4, 11]

Kuis Diagnostik & Latihan Mandiri

Q: Loop yang minimal dieksekusi 1 kali?

Q: Perintah untuk melompat ke iterasi berikutnya?

Q: Hasil dari perhitungan $10 \% 4$?

Q: `i<-1; while(i<5) {i<-i+1}`.
Berapa kali eksekusi?

`i<-1; while(i<5) {i<-i+1}`

Q: Penyebab `while(x>0)` loop selamanya jika `x=3`?

`while(x>0)`

Q: `for(i in c(2,4,6)) {s <- s+i}`.
Jika `s=0` awal, hasil akhir?

`for(i in c(2,4,6)) {s <- s+i}`

Langkah Selanjutnya: Latihan Mandiri

1. Ubah blok `for loop` menjadi `while loop`.

2. Modifikasi fungsi untuk menghitung total banyaknya bilangan ganjil.

3. Buat `repeat` untuk kelipatan 3 yang berhenti di angka 20.

4. Coba penentuan ganjil-genap tanpa loop sama sekali menggunakan struktur `ifelse()`.

Kuis Diagnostik & Latihan Mandiri

Q: Loop yang minimal dieksekusi 1 kali?

A: **REPEAT**

Q: Perintah untuk melompat ke iterasi berikutnya?

A: **next**

Q: Hasil dari perhitungan $10 \% 4$?

A: 2 (sisanya)

Q: $i \leftarrow -1$; **while**($i < 5$) { $i \leftarrow i+1$ }.
Berapa kali eksekusi?

$i \leftarrow -1$; **while**($i < 5$) { $i \leftarrow i+1$ }

A: 4 kali

Q: Penyebab **while**($x > 0$) loop selamanya jika $x=3$?

while($x > 0$)

A: x tidak pernah di-update di dalam loop.

Q: **for**(i in $c(2,4,6)$) { $s \leftarrow s+i$ }.
Jika $s=0$ awal, hasil akhir?

for(i in $c(2,4,6)$) { $s \leftarrow s+i$ }

A: Output: 12

Langkah Selanjutnya: Latihan Mandiri

1. Ubah blok **for loop** menjadi **while loop**.

2. Modifikasi fungsi untuk menghitung total banyaknya bilangan ganjil.

3. Buat **repeat** untuk kelipatan 3 yang berhenti di angka 20.

4. Coba penentuan ganjil-genap tanpa loop sama sekali menggunakan struktur **ifelse()**.